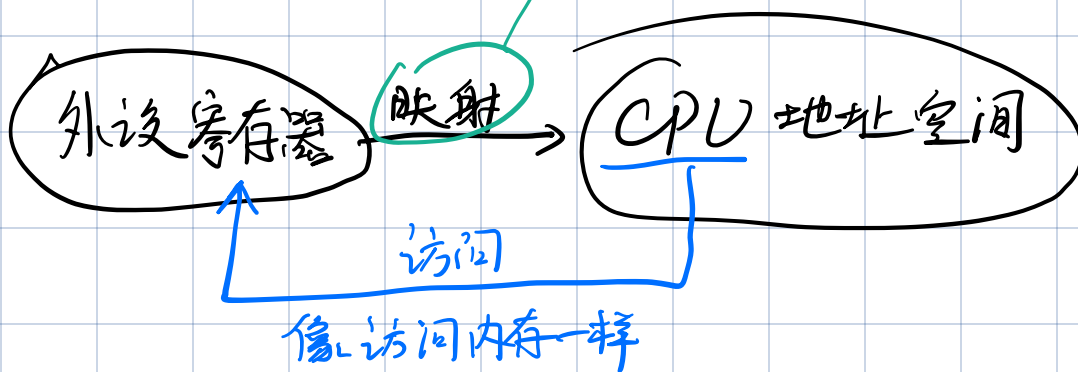


6.1

(reg) 寄存器映射

内存映射I/O: reg 占用内存空间
指针访问
端口映射I/O: 专用指令访问



意义: 让软件直接控制硬件外设
简化驱动设计

C语言访问寄存器

① 指针访问

$0x$ 开头代表十六进制数

#define GPIOA (0x40010800) STM32P103 固定 GPIO 地址

// 偏移地址

GPIO: 通用输入输出

#define GPIOA_ODR (*(volatile unsigned int *) (GPIOA + 0x0C))

强, 弱或带 volatile 声明的指针

解引用

// 点亮 LED

void LBD_on ()

{

GPIOA_ODR |= (1 << 5);

3

按位或

将二进制1左移5位

0010 0000

对应第5引脚

GPIOA 是 A 组通用输入输出端口

相当于一幢楼，0x40010800 是其基地址

通过基地址和偏移量 (offset) 找到楼中具体的房间地址，用 volatile unsigned int* 指针指向房间地址
再解引用房间地址得到其内容，房间即各种不同功能的寄存器
如 ODR (output data register)

当其第5位为1时，PA5 输出高电平，LED亮

② 结构体映射 (便于管理多个寄存器)

```
typedef struct {
    volatile unsigned int CRL;
    volatile unsigned int ODR;
} GPIO_Type;
```

```
#define GPIOA ((GPIO_Type*) 0x40010800)
```

```
int main ( )
```

```
{
```

```
    GPIOA->CRL = 0x00000001; // PA1 为输出模式
```

```
    GPIOA->ODR = 0x00000001; // PA1 输出高电平
```

```
    while(1);
}
```

6.2

时钟频率: 协调电子元件的每一步操作, 单位Hz
└─ 越高, 运算越快

主频 (SYSCLK)

分频

- CPU核心的运行频率
- 决定 CPU 运行速度 (计算能力)

HCLK (AHB 高级高性能总线时钟)

分频

- 决定内存访问速度

PClk1 (低速外设时钟)

PClk2 (高速外设时钟)

: 决定 APB 外设操作频率
高级外设总线

6.3

NVIC 中断控制器

Nested Vector Interrupt Controller
嵌套 向量 中断 控制器

作用: 管理 STM32 所有中断源

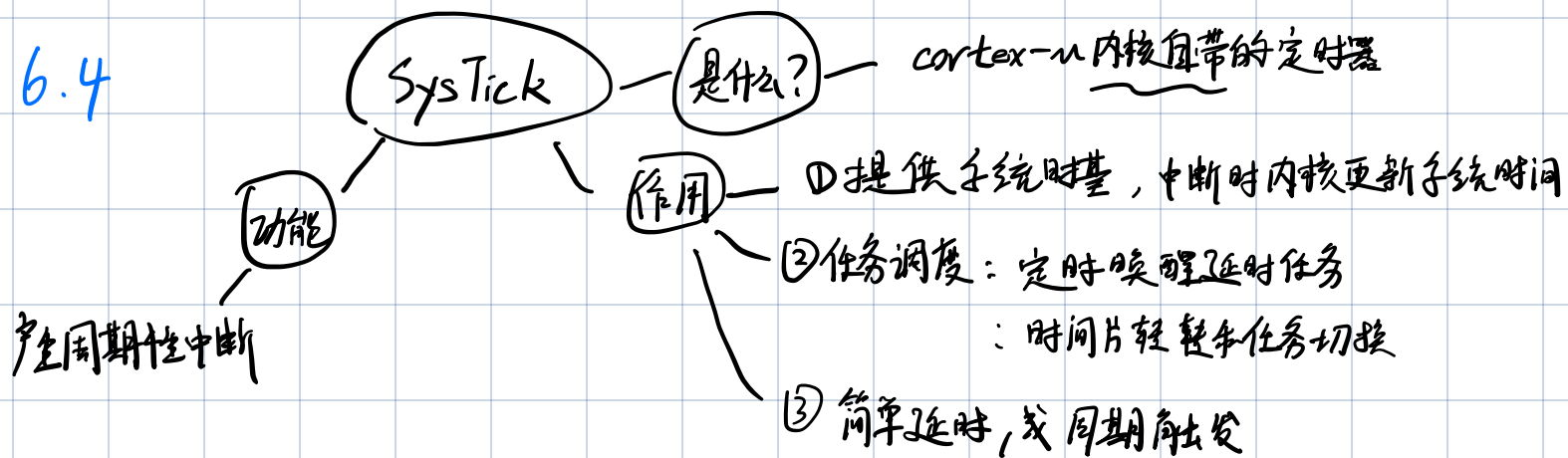
- 支持中断嵌套

核心机制: ① 抢占优先级
② 响应优先级

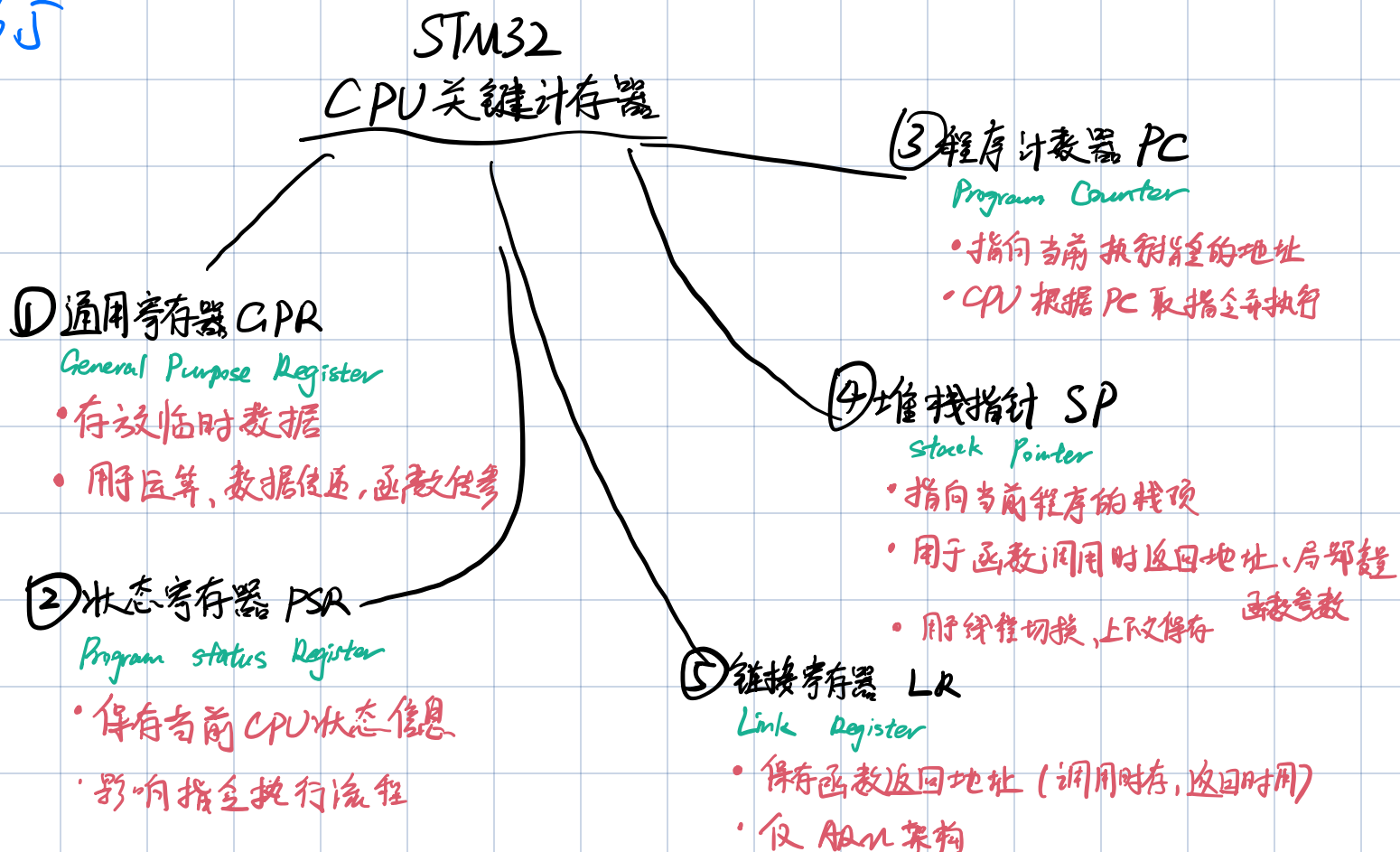
- 抢优决定是否嵌套，抢高打断抢低
- 同抢不嵌套，高响先行低抢后
- 同抢同响看中断号决定谁先

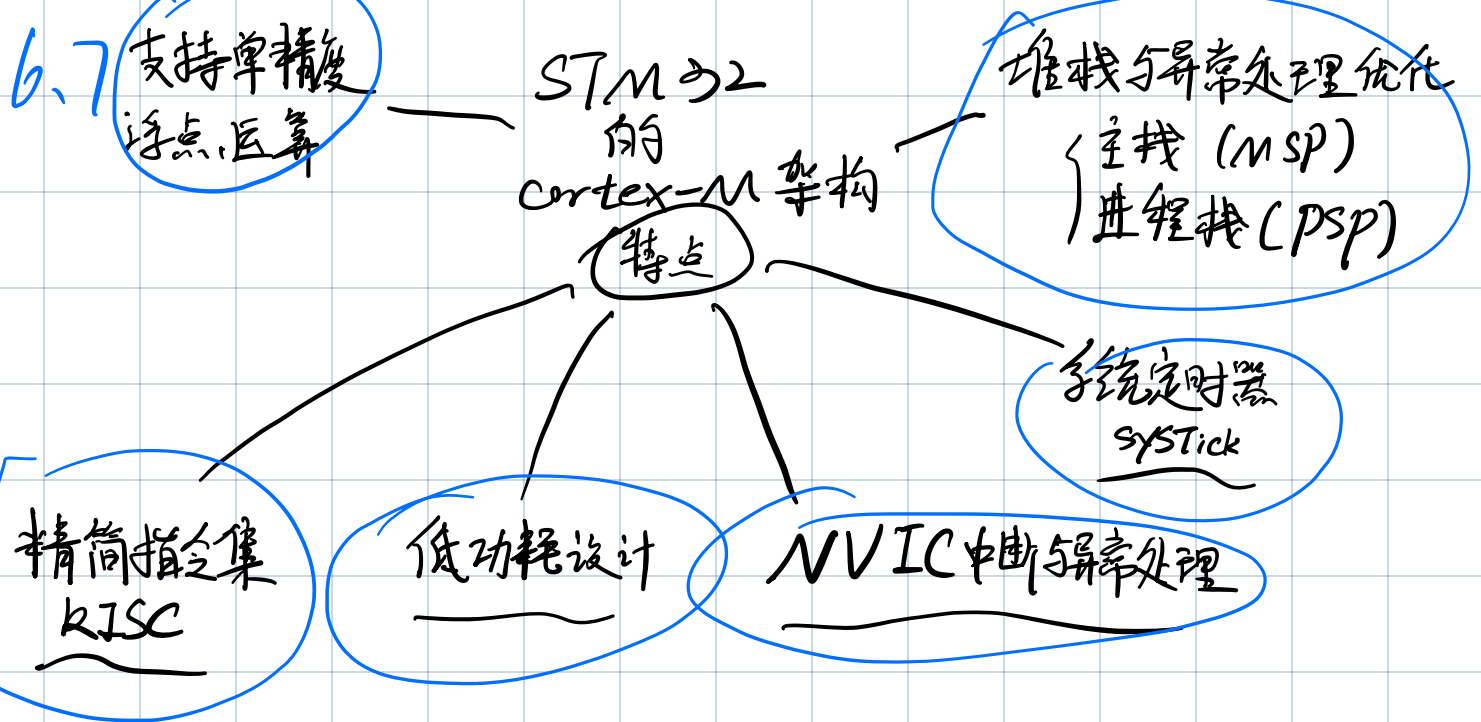
配置时，低数值 = 高优先级

6.4



6.5





6.8

STM32 启动文件:

Startup_stm32xxx.S

作用

- ① SP
- ↓
- ② 中断向量表
- ↓
- ③ Reset_Handler()
- ↓
- ④ 内存
- ↓
- ⑤ SystemInit()
- ↓
- ⑥ main()
- ↓
- ⑦ ISR

① 初始化堆栈指针 SP

↓

② 设置中断向量表

↓

③ 复位处理函数 Reset_Handler()

↓

④ 初始化内存

↓

⑤ 跳转 main()

↓

⑥ 定义中断和异常处理函数入口

STM32从(上电复位)到(进入main)的步骤:

① 上电复位



② 初始化SP



③ 跳转 Reset_Handler()

复位中断处理函数



④ 初始化 .data 段



⑤ 清零 .bss 段



⑥ 调用初始化函数 SystemInit()



⑦ 调用 main()

6.9 中断向量表

存放中断服务程序入口地址的表格

(作用) ① 定位 ISR 函数

② 快速响应

③

② 系统初始化的关键 (Reset_Handler 也在这)

特点

① 每个中断源对应一项

② 指针数组

③ cortex-m 中, 从 0x00000000 开始
| 第一项是初始堆栈指针值
| 第二项是 Reset_Handler

6.10

system Init() 系统初始化函数

作用

硬件相关的初始化配置

主要任务

① 配置系统时钟

② 初始化外设时钟

③ 配置闪存

④ 配置系统低功耗设置

位置 - 启动文件中

调用时机

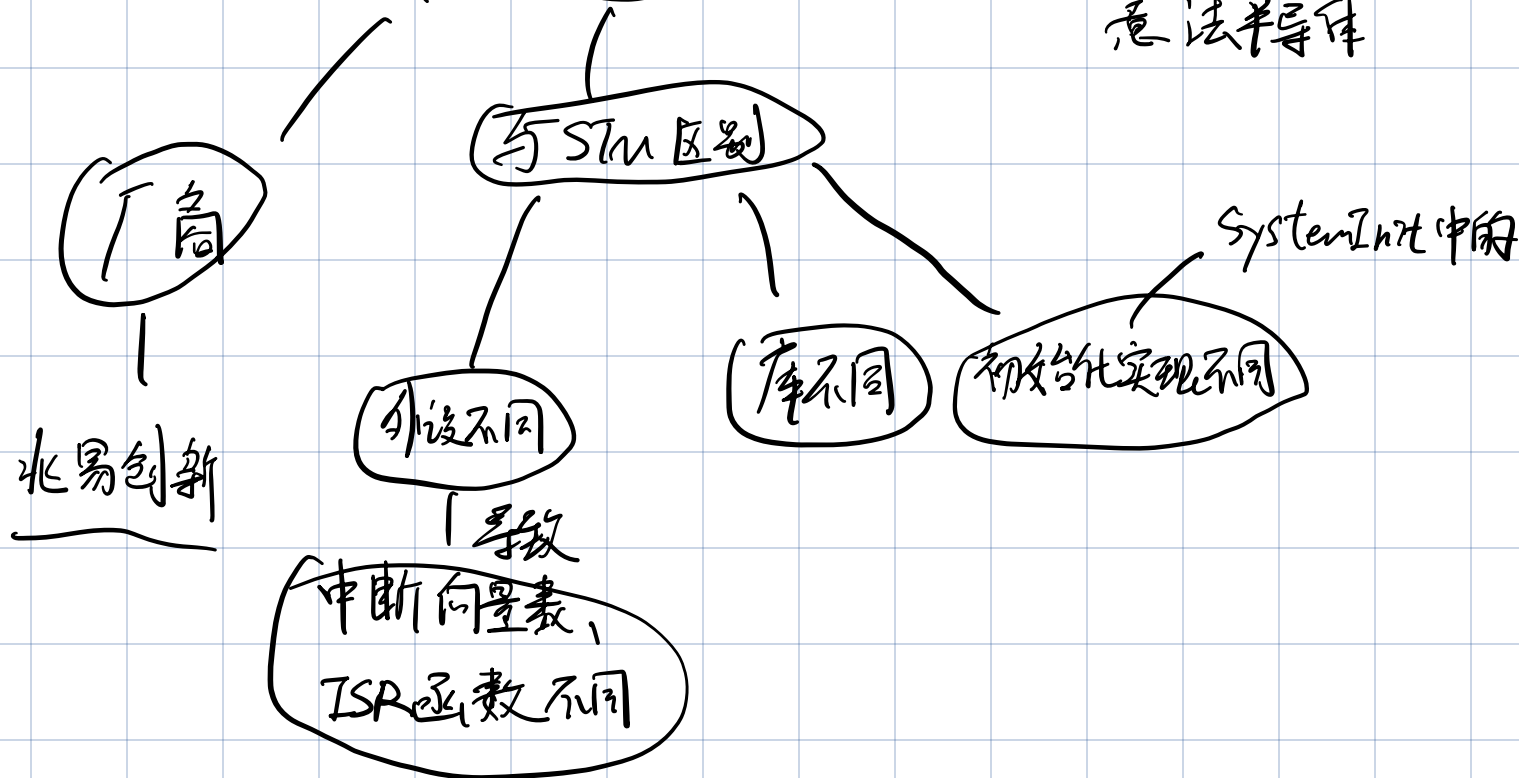
Reset_Handler() 调用
system Init() 后再调用 main()

6.11

GID32?

STM32

意法半导体



6.12

MSP 主堆栈指针

Main Stack Pointer

模式/场景 内核态下使用

空间用途 中断 异常处理

初始化时机 MCU上电复位时

切换机制 进入内核态自动切换

PSP 进程堆栈指针

Process Stack Pointer

用户态使用

普通线程/用户任务

切换到用户态时

调度器/程序显式切换

6.13

为什么要有MSP、PSP两个栈指针

① 分离任务处理和异常/中断处理的堆栈

② 稳定、安全

③ 支持多任务操作系统

任务^{平时} → PSP → 每个任务都有各自空间

↓
异常^{切换} → MSP → 避免任务间干扰

6.14

启动首个任务前:

要获取MSP初始值

目的

① 准确保存系统初始堆栈状态

② 保证异常和调度机制正常工作

③ 为了正确切换到用户态的PSP堆栈

6.15

GPIO工作模式

① 输入模式

② 输出模式

③ 复用功能模式：GPIO作为外设功能引脚

Alternative Function

- AF推挽：主动输出高/低电平
SPI, UART, CAN 等高速信号
- AF开漏：只能拉低，需上拉
才能输出高电平，I2C总线

④ 模拟模式：ADC输入, DAC输出
禁止数字信号输入

6.16

常用外设

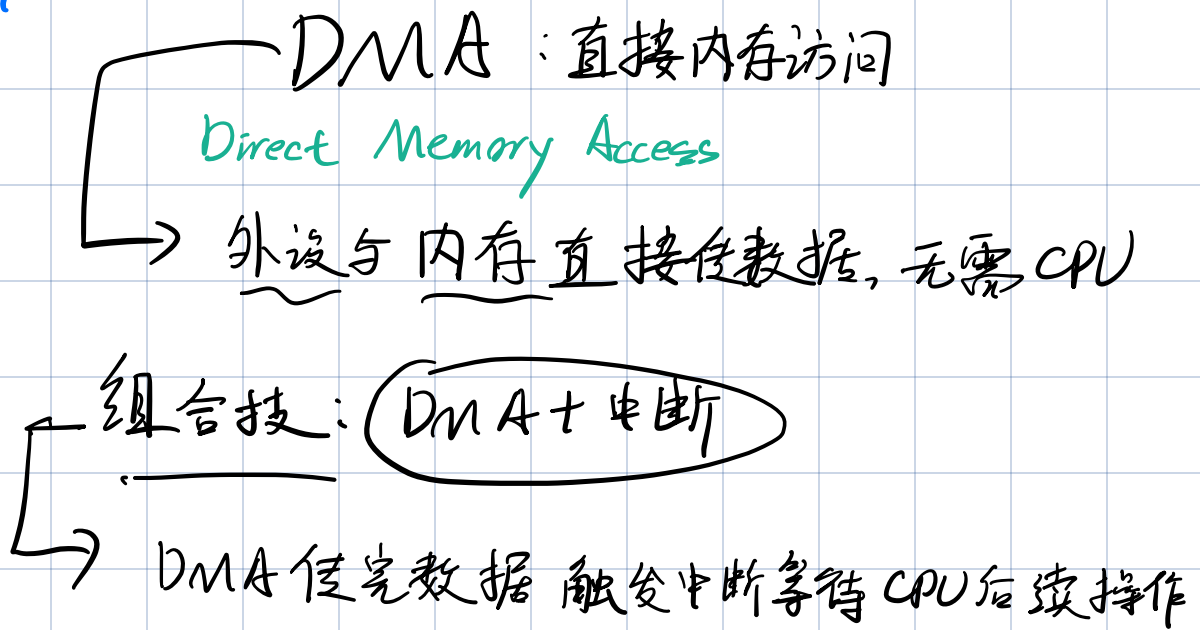
① ADC 模数转换器

② DAC

③ 定时器/计数器

- ④ ^{异步收发器 同步+异步收发器} UART / USART $\Rightarrow$ PC端调试
串行异步/同步通信接口 $\Rightarrow$ 连接模块(GPS, 蓝牙)
• 简单协议输入
- ⑤ SPI : 串行外设接口
Serial Peripheral Interface $\Rightarrow$ Flash, DAC, ADC, 传感器
• 高速全双工 同步串行通信
• 高速短距离
- ⑥ I2C : 集成电路间串行通信 $\Rightarrow$ 低速外设通信(温度湿度)
Inter-Integrated Circuit $\Rightarrow$ 板内通信

6.17



场景:

- ADC 采集
- DAC 输出
- USART/SPI 发送
- USB 内存搬运

6.18

UART + 中断

VS

DMA + 中断

场景 {
小数据量
实现简单
高实时性

{
高数据量
连续通信
降低 CPU 占用

6.19

内部看门狗 IWDG
Independant Watchdog

外部看门狗 WWDG
Window Watchdog

时钟源

内部低速独立时钟 LSI

PCLK / HCLK 时钟

特点

- 独立于系统主时钟, CPU 挂死 停机也能工作
- 配置后无法软件关闭

- 可设置时间窗口
- 必须在窗口内刷新, 否则复位
- 可被软件关闭

应用

- 用于系统硬件故障、CPU 停机 主时钟复位

- 捕获软件异常

6.20

STM32(低功耗模式)

① sleep 模式

- CPU停止, 外设运行
- 功耗: 高于 stop、standby
- 唤醒: 内部、外部中断
- 场景: 短时空闲等待

② stop 模式

- CPU和大部分外设停止
- 功耗: 低于 sleep
- 唤醒: 外部中断, RTC, 看门狗
- 场景: 长时空闲, 但需保持 SRAM

③ standby 模式

- CPU, 外设停止, 主时钟, PLL 停止, SRAM、寄存器内容丢失
- 功耗: 最低
- 唤醒: 外部中断(如按钮), RTC, IWDG
Real-Time-clock
闹钟功能
- 场景: 长时间休眠

6.21

Bootloader: 启动加载程序

什么时候用到 : 上电或复位后首先执行

存放位置 : 片内ROM或Flash

功能

- ① 初始化系统
- ② 加载用户程序
- ③ 固件升级
- ④ 系统恢复